Dokumentowe bazy danych – MongoDB

Ćwiczenie 2

Imiona i nazwiska autorów: Michał Kościanek, Michał Mąka

Odtwórz z backupu bazę `north0`

- najprostsza wersja

```
mongorestore dump
```

- to polecenie odtworzy wszystkie bazy danych znajdujące się we wskazanym folderze (w tym przypadku `dump`)
  - najłatwiej wgrać tam folder zawierający pliki z backupem i wykonać proste polecenie mongorestore
- dokumentacja:
  - https://www.mongodb.com/docs/database-tools/mongorestore/

Wybierz bazę north0

Baza `north0` jest kopią relacyjnej bazy danych `Northwind`

- poszczególne kolekcje odpowiadają tabelom w oryginalnej bazie `Northwind`

Zadanie 0

zapoznaj się ze strukturą dokumentów w bazie `North0`

```
db.customers.find()
db.orders.find();
db.orderdetails.find();
```

Zadanie 1 - operacje wyszukiwania danych, przetwarzanie dokumentów

Zadanie 1 - rozwiązania - Michał Mąka, code review & help - Michał Kościanek

a)

stwórz kolekcję `OrdersInfo` zawierającą następujące dane o zamówieniach

- kolekcję `OrdersInfo` należy stworzyć przekształcając dokumenty w oryginalnych kolekcjach `customers, orders, orderdetails, employees, shippers, products, categories, suppliers` do kolekcji w której pojedynczy dokument opisuje jedno zamówienie

```
db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "CustomerID",
      foreignField: "CustomerID",
      as: "Customer"
    }
  },
  { $unwind: "$Customer" },
  {
    $lookup: {
      from: "employees",
      localField: "EmployeeID",
      foreignField: "EmployeeID",
      as: "Employee"
    }
  },
  { $unwind: "$Employee" },
  {
    $lookup: {
      from: "shippers",
      localField: "ShipVia",
      foreignField: "ShipperID",
      as: "Shipper"
    }
  },
  { $unwind: "$Shipper" },

  {
    $lookup: {
      from: "orderdetails",
      localField: "OrderID",
      foreignField: "OrderID",
      as: "Orderdetails"
    }
  },
  { $unwind: "$Orderdetails" },

  {
    $lookup: {
      from: "products",
```

```
        localField: "Orderdetails.ProductID",
        foreignField: "ProductID",
        as: "Orderdetails.Product"
// zamiast osobno wydzielać, dodajemy do OrderDetails
      }
    },
    { $unwind: "$Orderdetails.Product" },
    {
      $lookup: {
        from: "categories",
        localField: "Orderdetails.Product.CategoryID",
        foreignField: "CategoryID",
        as: "Orderdetails.Product.Category"
      }
    },
    { $unwind: "$Orderdetails.Product.Category" },

    {
      $lookup: {
        from: "suppliers",
        localField: "Orderdetails.Product.SupplierID",
        foreignField: "SupplierID",
        as: "Orderdetails.Product.Supplier"
      }
    },
    { $unwind: "$Orderdetails.Product.Supplier" },

    // zgrupowanie wszystkich danych
    {
      $group: {
        _id: "$_id",
// stosujemy first, bo te OrderID sie powtarzają (unwind)
        OrderID: { $first: "$OrderID" },
          Dates: {
            $first: {
              OrderDate: "$OrderDate",
              RequiredDate: "$RequiredDate",
              /*ShippedDate: "$ShippedDate" */ }
          },
        Customer: { $first: {
          CustomerID: "$Customer.CustomerID",
          CompanyName: "$Customer.CompanyName",
          City: "$Customer.City",
          Country: "$Customer.Country"
        }},
        Employee: { $first: {
              EmployeeID: "$Employee.EmployeeID",
              FirstName: "$Employee.FirstName",
              LastName: "$Employee.LastName",
              Title: "$Employee.Title"
        }},
        Shipment: { $first: {
          "Shipper": {
              "ShipperID":  "$shipper.ShipperID",
```

```
            "CompanyName": "$shipper.CompanyName"
            },
        "ShipName": "$ShipName",
        "ShipAddress": "$ShipAddress",
        "ShipCity": "$ShipCity",
        "ShipCountry": "$ShipCountry"
        },
      },
      Orderdetails: {
      //oddzielne dopisujemy
        $push: {
          UnitPrice: "$Orderdetails.UnitPrice",
          Quantity: "$Orderdetails.Quantity",
          Discount: "$Orderdetails.Discount",
          Value: "$Orderdetails.Value",
          Product: {
            ProductID: "$Orderdetails.ProductID",
            ProductName: "$Orderdetails.ProductName",
            QuantityPerUnit: "$Orderdetails.QuantityPerUnit",
            CategoryID: "$Orderdetails.Product.CategoryID",
            CategoryName: "$Orderdetails.Product.CategoryName",
            },
        }
      },
      OrderTotal: {
        $sum: {
          $multiply: [
            "$Orderdetails.UnitPrice",
            "$Orderdetails.Quantity",
            { $subtract: [1, "$Orderdetails.Discount"] }
          ]
        }
      },
      Freight: { $first: "$Freight"}
    }
  },
  { $out: "OrdersInfo" },
]);
```

spodziewany wynik:

```
[
  {
    "_id": ...

    OrderID": ... numer zamówienia

    "Customer": {  ... podstawowe informacje o kliencie skladającym
      "CustomerID": ... identyfikator klienta
      "CompanyName": ... nazwa klienta
      "City": ... miasto
```

```
      "Country": ... kraj
    },

    "Employee": {  ... podstawowe informacje o pracowniku obsługującym
zamówienie
      "EmployeeID": ... idntyfikator pracownika
      "FirstName": ... imie
      "LastName": ... nazwisko
      "Title": ... stanowisko

    },

    "Dates": {
       "OrderDate": ... data złożenia zamówienia
       "RequiredDate": data wymaganej realizacji
    }

    "Orderdetails": [  ... pozycje/szczegóły zamówienia - tablica takich
pozycji
      {
        "UnitPrice": ... cena
        "Quantity": ... liczba sprzedanych jednostek towaru
        "Discount": ... zniżka
        "Value": ... wartośc pozycji zamówienia
        "product": { ... podstawowe informacje o produkcie
          "ProductID": ... identyfikator produktu
          "ProductName": ... nazwa produktu
          "QuantityPerUnit": ... opis/opakowannie
          "CategoryID": ... identyfikator kategorii do której należy
produkt
          "CategoryName" ... nazwę tej kategorii
        },
      },
      ...
    ],

    "Freight": ... opłata za przesyłkę
    "OrderTotal"  ... sumaryczna wartosc sprzedanych produktów

    "Shipment" : {  ... informacja o wysyłce
        "Shipper": { ... podstawowe inf o przewoźniku
           "ShipperID":
            "CompanyName":
        }
        ... inf o odbiorcy przesyłki
        "ShipName": ...
        "ShipAddress": ...
        "ShipCity": ...
        "ShipCountry": ...
    }
  }
]
```

wynik


```
[
  {
    "_id": {"$oid": "63a060b9bb3b972d6f4e208b"},
    "Customer": {
      "CustomerID": "BERGS",
      "CompanyName": "Berglunds snabbköp",
      "City": "Luleå",
      "Country": "Sweden"
    },
    "Dates": {
      "OrderDate": {"$date": "1997-02-13T00:00:00.000Z"},
      "RequiredDate": {"$date": "1997-03-13T00:00:00.000Z"}
    },
    "Employee": {
      "EmployeeID": 3,
      "FirstName": "Janet",
      "LastName": "Leverling",
      "Title": "Sales Representative"
    },
    "Freight": 9.3,
    "OrderID": 10445,
    "OrderTotal": 174.9,
    "Orderdetails": [
      {
        "UnitPrice": 14.4,
        "Quantity": 6,
        "Discount": 0,
        "Product": {
          "ProductID": 39,
          "CategoryID": 1
        }
      },
      {
        "UnitPrice": 5.9,
        "Quantity": 15,
        "Discount": 0,
        "Product": {
          "ProductID": 54,
          "CategoryID": 6
        }
      }
    ],
    "ShipAddress": "Berguvsvägen  8",
    "Shipment": {
      "Shipper": {
      },
      "ShipName": "Berglunds snabbköp",
      "ShipAddress": "Berguvsvägen  8",
      "ShipCity": "Luleå",
      "ShipCountry": "Sweden"
```

```
      }
    }
  ]
```

b)

stwórz kolekcję `CustomerInfo` zawierającą następujące dane każdym kliencie

- pojedynczy dokument opisuje jednego klienta

```
db.customers.aggregate([
  {
    $lookup: {
      from: "OrdersInfo",
      localField: "CustomerID",
      foreignField: "Customer.CustomerID",
      as: "Orders"
    }
  },

  {
    $project: {
      _id: 1,
      CustomerID: 1,
      CompanyName: 1,
      City: 1,
      Country: 1,
      // wybieramy to co nam potrzeba
      Orders: {
        $map: {
          input: "$Orders",
          as: "order",
          in: {
            // $$ - odniesienie do zmiennej (przez map)
            OrderID: "$$order.OrderID",
            Dates: "$$order.Dates",
            Employee: "$$order.Employee",
            Orderdetails: "$$order.Orderdetails",
            Freight: "$$order.Freight",
            OrderTotal: "$$order.OrderTotal",
            ShipAddress: "$$order.ShipAddress",
            Shipment: "$$order.Shipment"
          }
        }
      }
    }
  },
```

```
  { $out: "CustomerInfo" }
]);
```

spodziewany wynik:

```
[
  {
    "_id": ...

    "CustomerID": ... identyfikator klienta
    "CompanyName": ... nazwa klienta
    "City": ... miasto
    "Country": ... kraj

    "Orders": [ ... tablica zamówień klienta o strukturze takiej jak w
punkcie a)
                    (oczywiście bez informacji o kliencie)
    ]
]
```

wynik

```
[
  {
    "_id": {"$oid": "63a05cdfbb3b972d6f4e097b"},
    "City": "Berlin",
    "CompanyName": "Alfreds Futterkiste",
    "Country": "Germany",
    "CustomerID": "ALFKI",
    "Orders": [
      {
        "OrderID": 10643,
        "Dates": {
          "OrderDate": {"$date": "1997-08-25T00:00:00.000Z"},
          "RequiredDate": {"$date": "1997-09-22T00:00:00.000Z"}
        },...
        // wynik ma 270 linii kodu (dla limit(1))
```

c)

Napisz polecenie/zapytanie: Dla każdego klienta pokaż wartość zakupionych przez niego produktów z kategorii 'Confections' w 1997r

- Spróbuj napisać to zapytanie wykorzystując

  - oryginalne kolekcje (`customers, orders, orderdertails, products, categories`)
  - kolekcję `OrderInfo`
  - kolekcję `CustomerInfo`

- porównaj zapytania/polecenia/wyniki

```
[
  {
    "_id":

    "CustomerID": ... identyfikator klienta
    "CompanyName": ... nazwa klienta
    "ConfectionsSale97": ... wartość zakupionych przez niego produktów
                             z kategorii 'Confections'  w 1997r

  }
]
```

rozwiazanie:

```
//oryginalne kolekcje (customers, orders, orderdetails, products,
categories)
db.customers.find()
db.orders.find()
db.orderdetails.find()

db.customers.aggregate([
  {
    $lookup: {
      from: "orders",
      let: { custId: "$CustomerID" }, // przypisanie do custId
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $eq: ["$CustomerID", "$$custId"] },
                { $eq: [{ $year: "$OrderDate" }, 1997] }
              ]
            }
          }
        },
        { $lookup: { from: "orderdetails", localField: "OrderID",
          foreignField: "OrderID", as: "details" } },
        { $unwind: "$details" },
        { $lookup: { from: "products", localField: "details.ProductID",
          foreignField: "ProductID", as: "product" } },
        { $unwind: "$product" },
```

```
          { $lookup: { from: "categories", localField: "product.CategoryID",
            foreignField: "CategoryID", as: "category" } },
          { $unwind: "$category" },
          // filtr na confections
          { $match: { "category.CategoryName": "Confections" } },
          {
            $project: {
              value: {
                $multiply: [
                  "$details.UnitPrice",
                  "$details.Quantity",
                  { $subtract: [1, "$details.Discount"] }
                ]
              }
            }
          }
        ],
        as: "orders_with_conf_1997"
      }
    },
    {
      $project: {
        _id: 1,
        CustomerID: 1,
        CompanyName: 1,
        ConfectionsSale97: { $sum: "$orders_with_conf_1997.value" }
      }
    },
  ]);

  db.orderdetails.find().limit(1)
  db.OrdersInfo.find().limit(1)

  //1c wykorzystując OrderInfo

  db.OrdersInfo.aggregate([
    {
      $unwind: {
        path: "$Orderdetails",
      }
    },
    {
      $group: {
        _id: "$Customer.CustomerID",
        CompanyName: { $first: "$Customer.CompanyName" },
        ConfectionsSale97: {
          $sum: {
            $cond: [
              // if
              {
                $and: [
                  { $gte: ["$Dates.OrderDate", ISODate("1997-01-01")] },
                  { $lt: ["$Dates.OrderDate", ISODate("1998-01-01")] },
                  { $eq: ["$Orderdetails.Product.CategoryName",
```

```
                    "Confections"] }
                ]
              },
              // true
              {
                $multiply: [
                  "$Orderdetails.UnitPrice",
                  "$Orderdetails.Quantity",
                  { $subtract: [1, "$Orderdetails.Discount"] }
                ]
              },
              // false
              0
            ]
          }
        }
      }
    },
    {
      $project: {
        _id: 1,
        CustomerID: "$_id",
        CompanyName: 1,
        ConfectionsSale97: 1
      }
    }
  ]);

  //1c wykorzystując CustomerInfo


  db.CustomerInfo.aggregate([
    {
      $unwind: {
        path: "$Orders",
        preserveNullAndEmptyArrays: true
      }
    },
    {
      $unwind: {
        path: "$Orders.Orderdetails",
        preserveNullAndEmptyArrays: true
      }
    },
    {
      $group: {
        _id: "$CustomerID",
        CompanyName: { $first: "$CompanyName" },
        ConfectionsSale97: {
          $sum: {
            $cond: [
              // if
              {
                $and: [
```

```
                  { $gte: ["$Orders.Dates.OrderDate", ISODate("1997-01-01")]
  },
                  { $lt: ["$Orders.Dates.OrderDate", ISODate("1998-01-01")]
  },
                  { $eq: ["$Orders.Orderdetails.Product.CategoryName",
                    "Confections"] }
                ]
              },
              // true
              {
                $multiply: [
                  "$Orders.Orderdetails.UnitPrice",
                  "$Orders.Orderdetails.Quantity",
                  { $subtract: [1, "$Orders.Orderdetails.Discount"] }
                ]
              },
              // false
              0
            ]
          }
        }
      }
    },
    {
      $project: {
        _id: 1,
        CustomerID: "$_id",
        CompanyName: 1,
        ConfectionsSale97: 1
      }
    }
  ]);
```

wynik:

dla wszystkich rozwiązań jest spójny wynik:

```
...  {
    "_id": {"$oid": "63a05cdfbb3b972d6f4e097e"},
    "CompanyName": "Around the Horn",
    "ConfectionsSale97": 375.19999977201223,
    "CustomerID": "AROUT"
  },...
```

UWAGA: wynik nie jest dokładnie taki sam w przypadku operowania na OrdersInfo, w wyniku nie znajdziemy takich klientów, którzy nie mają zamówień

d)

Napisz polecenie/zapytanie: Dla każdego klienta poaje wartość sprzedaży z podziałem na lata i miesiące Spróbuj napisać to zapytanie wykorzystując - oryginalne kolekcje (`customers, orders, orderdertails, products, categories`) - kolekcję `OrderInfo` - kolekcję `CustomerInfo`

- porównaj zapytania/polecenia/wyniki

```
// 1d - oryginalne kolekcje

db.customers.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "CustomerID",
      foreignField: "CustomerID",
      as: "Order"
    }
  },
  { $unwind: { path: "$Order", preserveNullAndEmptyArrays: true } },
    {
    $lookup: {
      from: "orderdetails",
      localField: "Order.OrderID",
      foreignField: "OrderID",
      as: "OrderDetail"
    }
  },
  { $unwind: { path: "$OrderDetail", preserveNullAndEmptyArrays: true } },
  {
    $lookup: {
      from: "products",
      localField: "OrderDetail.ProductID",
      foreignField: "ProductID",
      as: "Product"
    }
  },
  { $unwind: { path: "$Product", preserveNullAndEmptyArrays: true } },
    {
    $lookup: {
      from: "categories",
      localField: "Product.CategoryID",
      foreignField: "CategoryID",
      as: "Category"
    }
  },
  { $unwind: { path: "$Category", preserveNullAndEmptyArrays: true } },
    // grupowanie ze wzdlegu na rok i miesiąc
  {
    $group: {
      _id: {
        CustomerID: "$CustomerID",
```

```
          CompanyName: "$CompanyName",
          Year: { $year: "$Order.OrderDate" },
          Month: { $month: "$Order.OrderDate" }
        },
        MonthlyValue: {
          $sum: {
            $cond: [
              { $ifNull: ["$OrderDetail.UnitPrice", false] }, // czy istnieje
              {
                $multiply: [
                  "$OrderDetail.UnitPrice",
                  "$OrderDetail.Quantity",
                  { $subtract: [1, "$OrderDetail.Discount"] }
                ]
              },
              0 // brak zamowień
            ]
          }
        }
      }
    },

  // do jednego klienta
    {
      $group: {
        _id: "$_id.CustomerID",
        CompanyName: { $first: "$_id.CompanyName" },
        TotalLifetimeValue: { $sum: "$MonthlyValue" },
        SalesHistory: {
          $push: {
            Year: "$_id.Year",
            Month: "$_id.Month",
            Value: "$MonthlyValue"
          }
        }
      }
    },

    {
      $project: {
        _id: 0,
        CustomerID: "$_id",
        CompanyName: 1,
        TotalLifetimeValue: 1,
        SalesHistory: "$SalesHistory"
        }
      }
  ]);




  // 1d - OrdersInfo
```

```
db.OrdersInfo.aggregate([
  {
    $unwind: {
      path: "$Orderdetails",
      preserveNullAndEmptyArrays: true
    }
  },

    // group: year + month
  {
    $group: {
      _id: {
        CustomerID: "$Customer.CustomerID",
        CompanyName: "$Customer.CompanyName",
        Year: { $year: "$Dates.OrderDate" },
        Month: { $month: "$Dates.OrderDate" }
      },
      MonthlyValue: {
        $sum: {
          $cond: [
            { $ifNull: ["$Orderdetails.UnitPrice", false] },
            {
              $multiply: [
                "$Orderdetails.UnitPrice",
                "$Orderdetails.Quantity",
                { $subtract: [1, "$Orderdetails.Discount"] }
              ]
            },
            0 // nie ma
          ]
        }
      }
    }
  },
        // group: by customer
  {
    $group: {
      _id: "$_id.CustomerID",
      CompanyName: { $first: "$_id.CompanyName" },
      TotalLifetimeValue: { $sum: "$MonthlyValue" },
      SalesHistory: {
        $push: {
          Year: "$_id.Year",
          Month: "$_id.Month",
          Value: "$MonthlyValue"
        }
      }
    }
  },

  {
    $project: {
      _id: 0,
      CustomerID: "$_id",
```

```
      CompanyName: 1,
      TotalLifetimeValue: 1,
      SalesHistory: "$SalesHistory"
    }
  },
]);


// 1d - customerInfo

db.CustomerInfo.aggregate([
  {
    $unwind: {
      path: "$Orders",
      preserveNullAndEmptyArrays: true
    }
  },

  {
    $unwind: {
      path: "$Orders.Orderdetails",
      preserveNullAndEmptyArrays: true
    }
  },

  {
    $group: {
      _id: {
        CustomerID: "$CustomerID",
        CompanyName: "$CompanyName",
        Year: { $year: "$Orders.Dates.OrderDate" },
        Month: { $month: "$Orders.Dates.OrderDate" }
      },
      MonthlyValue: {
        $sum: {
          $cond: [
            { $ifNull: ["$Orders.Orderdetails.UnitPrice", false] },
            {
              $multiply: [
                "$Orders.Orderdetails.UnitPrice",
                "$Orders.Orderdetails.Quantity",
                { $subtract: [1, "$Orders.Orderdetails.Discount"] }
              ]
            },
            0
          ]
        }
      }
    }
  },

  {
    $group: {
      _id: "$_id.CustomerID",
```

```
        CompanyName: { $first: "$_id.CompanyName" },
        TotalLifetimeValue: { $sum: "$MonthlyValue" },
        SalesHistory: {
          $push: {
            Year: "$_id.Year",
            Month: "$_id.Month",
            Value: "$MonthlyValue"
          }
        }
      }
    },

    {
      $project: {
        _id: 0,
        CustomerID: "$_id",
        CompanyName: 1,
        TotalLifetimeValue: 1,
        SalesHistory: "$SalesHistory"
      }
    },
  ]);
```

Wyniki: (dla wszystkich + per customer)

```
    {
      "CompanyName": "Let's Stop N Shop",
      "CustomerID": "LETSS",
      "SalesHistory": [
        {
          "Year": 1998,
          "Month": 2,
          "Value": 1378.0699989192187
        },
        {
          "Year": 1997,
          "Month": 10,
          "Value": 844.2525
        },
        {
          "Year": 1997,
          "Month": 11,
          "Value": 536.3999991118908
        },
        {
          "Year": 1997,
          "Month": 6,
          "Value": 317.75
        }
      ],
```

```
      "TotalLifetimeValue": 3076.4724980311094
    },
    {
      "CompanyName": "FISSA Fabrica Inter. Salchichas S.A.",
      "CustomerID": "FISSA",
      "SalesHistory": [
        {
          "Year": null,
          "Month": null,
          "Value": 0
        }
      ],
      "TotalLifetimeValue": 0
    },
```

UWAGA: powyższy wynik, nie jest taki sam w przypadku operowania na OrdersInfo, wynik dla FISSA, jest nieobecny, ponieważ, nie znajduje się on w OrdersInfo (brak zamówień)

e)

Załóżmy że pojawia się nowe zamówienie dla klienta 'ALFKI', zawierające dwa produkty 'Chai' oraz "Ikura"

- pozostałe pola w zamówieniu (ceny, liczby sztuk prod, inf o przewoźniku itp. możesz uzupełnić wg własnego uznania) Napisz polecenie które dodaje takie zamówienie do bazy
- aktualizując oryginalne kolekcje `orders`, `orderdetails`
- aktualizując kolekcję `OrderInfo`
- aktualizując kolekcję `CustomerInfo`

Napisz polecenie

- aktualizując oryginalną kolekcję orderdetails
- aktualizując kolekcję `OrderInfo`
- aktualizując kolekcję `CustomerInfo`

dodawanie danych

```
db.products.find({ "ProductName": "Chai" }) // unitprice = 18
db.products.find({ "ProductName": "Ikura" }) // unitprice = 31

// 1e kolekcje (orders + orderdetails)
db.orders.insertOne({
  OrderID: 99999,
  CustomerID: "ALFKI",
  EmployeeID: 1,
  OrderDate: ISODate("2026-05-15"),
  RequiredDate: ISODate("2026-06-15"),
  ShipVia: 1,
```

```
    Freight: 15.50,
    ShipName: "Alfreds Futterkiste",
    ShipAddress: "Obere Str. 57",
    ShipCity: "Berlin",
    ShipCountry: "Germany"
  });

  db.orderdetails.insertMany([
    { OrderID: 99999, ProductID: 1, UnitPrice: 18.00,
        Quantity: 10, Discount: 0 },   // Chai

    { OrderID: 99999, ProductID: 10, UnitPrice: 31.00,
        Quantity: 5, Discount: 0 }    // Ikura
  ]);

  db.customers.find({ "CustomerID": "ALFKI" })
  db.employees.find({ "EmployeeID": 1 })


  // 1e OrdersInfo
  db.OrdersInfo.insertOne({
    OrderID: 99999,
    Customer: {
      CustomerID: "ALFKI",
      CompanyName: "Alfreds Futterkiste",
      City: "Berlin",
      Country: "Germany"
    },
    Dates: {
      OrderDate: ISODate("2026-05-15"),
      RequiredDate: ISODate("2026-06-15")
    },
    Employee: { EmployeeID: 1, FirstName: "Nancy",
        LastName: "Davolio", Title: "Sales Representative" },
    Freight: 15.50,
    OrderTotal: 335.00, // 18*10+31*5
    Shipment: {
      Shipper: { ShipperID: 1, CompanyName: "Speedy Express" },
      ShipName: "Alfreds Futterkiste",
      ShipAddress: "Obere Str. 57",
      ShipCity: "Berlin",
      ShipCountry: "Germany"
    },
    Orderdetails: [
      {
        UnitPrice: 18.00, Quantity: 10, Discount: 0,
        Product: { ProductID: 1, ProductName: "Chai",
            CategoryID: 1, CategoryName: "Beverages" }
      },
      {
        UnitPrice: 31.00, Quantity: 5, Discount: 0,
        Product: { ProductID: 10, ProductName: "Ikura",
            CategoryID: 8, CategoryName: "Seafood" }
      }
```

```
    ]
  });


  // 1e customerInfo

  db.CustomerInfo.updateOne(
    { CustomerID: "ALFKI" },
    {
      $push: { // dodaj do tablicy Orders klienta
        Orders: {
          OrderID: 99999,
          Dates: { OrderDate: ISODate("2026-05-15T00:00:00Z"),
                   RequiredDate: ISODate("2026-06-15T00:00:00Z") },
          Employee: { EmployeeID: 1, FirstName: "Nancy",
                      LastName: "Davolio", Title: "Sales Representative" },
          Freight: 15.50,
          OrderTotal: 335.00,
          Shipment: {
              Shipper: { ShipperID: 1, CompanyName: "Speedy Express" },
              ShipName: "Alfreds Futterkiste",
              ShipAddress: "Obere Str. 57",
              ShipCity: "Berlin",
              ShipCountry: "Germany"
            },
          Orderdetails: [
            { UnitPrice: 18.00, Quantity: 10, Discount: 0,
              Product: { ProductID: 1, ProductName: "Chai" }},
            { UnitPrice: 31.00, Quantity: 5, Discount: 0,
              Product: { ProductID: 10, ProductName: "Ikura" }}
          ]
        }
      }
    }
  );
```

edycja danych

```
  // aktualizacja 1e kolekcje
  db.orderdetails.updateOne(
    { OrderID: 99999, ProductID: 1 },
    { $set: { Quantity: 20 } }
  );

  // aktualizacja 1e OrdersInfo

  db.OrdersInfo.updateOne(
    { OrderID: 99999 },
    {
```

```
      $set: {
        "Orderdetails.$[elem].Quantity": 20, // dla elem
        "OrderTotal": 515.00      // suma sie zmienia
      }
    },
    { // przypisanie
      arrayFilters: [ { "elem.Product.ProductID": 1 } ]   // := elem
    }
  );

  // aktualizacja 1e CustomerInfo

  db.CustomerInfo.updateOne(
    { CustomerID: "ALFKI" },
    {
      $set: {
        // "ord" to element tablicy Orders, a "prod" to element Orderdetails
        "Orders.$[ord].Orderdetails.$[prod].Quantity": 20,
        "Orders.$[ord].OrderTotal": 515.00 // suma sie zmienia
      }
    },
    {
      arrayFilters: [ // przypisania
        { "ord.OrderID": 99999 },       // := ord (orderid = 99999)
        { "prod.Product.ProductID": 1 } // := prod (prodid = 1)
      ]
    }
  );
```

Wynik

```
  // np. OrdersInfo
  [
    {
      "_id": {"$oid": "6a067a0940cc6835962cdab3"},
      "Customer": {
        "CustomerID": "ALFKI",
        "CompanyName": "Alfreds Futterkiste",
        "City": "Berlin",
        "Country": "Germany"
      },
      "Dates": {
        "OrderDate": {"$date": "2026-05-15T00:00:00.000Z"},
        "RequiredDate": {"$date": "2026-06-15T00:00:00.000Z"}
      },
      "Employee": {
        "EmployeeID": 1,
        "FirstName": "Nancy",
        "LastName": "Davolio",
        "Title": "Sales Representative"
```

```
      },
      "Freight": 15.5,
      "OrderID": 99999,
      "OrderTotal": 515,
      "Orderdetails": [
        {
          "UnitPrice": 18,
          "Quantity": 20,
          "Discount": 0,
          "Product": {
            "ProductID": 1,
            "ProductName": "Chai",
            "CategoryID": 1,
            "CategoryName": "Beverages"
          }
        },
        {
          "UnitPrice": 31,
          "Quantity": 5,
          "Discount": 0,
          "Product": {
            "ProductID": 10,
            "ProductName": "Ikura",
            "CategoryID": 8,
            "CategoryName": "Seafood"
          }
        }
      ],
      "Shipment": {
        "Shipper": {
          "ShipperID": 1,
          "CompanyName": "Speedy Express"
        },
        "ShipName": "Alfreds Futterkiste",
        "ShipAddress": "Obere Str. 57",
        "ShipCity": "Berlin",
        "ShipCountry": "Germany"
      }
    }
  ]
```

porównanie:

Odczytywanie danych z kolekcji `CustomerInfo` lub `OrdersInfo` jest łatwe i wygodne, ale dodawanie, a szczególnie modifikowanie danych to katastrofa, dodatkowo łatwo jest coś zgubić.

f)

Napisz polecenie które modyfikuje zamówienie dodane w pkt e) zwiększając zniżkę o 5% (dla każdej pozycji tego zamówienia)

Napisz polecenie

- aktualizując oryginalną kolekcję `orderdetails`
- aktualizując kolekcję `OrderInfo`
- aktualizując kolekcję `CustomerInfo`

```
// 1f kolekcje

db.orderdetails.updateMany(
  { OrderID: 99999 },
  { $inc: { Discount: 0.05 } } //zwiększ o
);

// 1f OrdersInfo
db.OrdersInfo.updateOne(
  { OrderID: 99999 },
  [ // [] Aggregation Pipeline

  // + 5% discount
    {
      $set: {
        Orderdetails: {
          $map: {
            input: "$Orderdetails",
            as: "item",
            in: {
              // $mergeObjects nadpisuje tylko discount, reszte zostawia
              $mergeObjects: [
                "$$item",
                { Discount: { $add: ["$$item.Discount", 0.05] } }
              ]
            }
          }
        }
      }
    },

    // musimy przeliczyc OrderTotal
    {
      $set: {
        OrderTotal: {
          $sum: {
            $map: {
              input: "$Orderdetails", // to jest już ta NOWA tablica
              as: "item",
              in: {
                $multiply: [
                  "$$item.UnitPrice",
                  "$$item.Quantity",
                  { $subtract: [1, "$$item.Discount"] }
```

```
                ]
              }
            }
          }
        }
      }
    }
  ]
);

// aktualizacja discount, podzielona na 2, bo to jakas masakra z pipeline
// 1. discount
db.CustomerInfo.updateOne(
  { CustomerID: "ALFKI" }, // Znajdź klienta
  {
    $inc: {
      // w [ord] dla wszystkich produktów ($[]), inc. 0.05
      "Orders.$[ord].Orderdetails.$[].Discount": 0.05
    }
  },
  {
    arrayFilters: [
      { "ord.OrderID": 99999 } // := ord
    ]
  }
);

// 2. aktualizacja OrderTotal
db.CustomerInfo.updateOne(
  { CustomerID: "ALFKI", "Orders.OrderID": 99999 },
  [
    {
      $set: {
        Orders: {
          $map: {
            input: "$Orders",
            as: "order",
            in: {
              $cond: {
                if: { $eq: ["$$order.OrderID", 99999] },
          // Jeśli to szukane zamówienie, nadpisujemy tylko OrderTotal
                then: {
                  $mergeObjects: [
                    "$$order",
                    {
                    // wyliczenie nowego OrderTotal
                      OrderTotal: {
                        $sum: {
                          $map: {
                            input: "$$order.Orderdetails", // po zmianie
                            as: "prod",
                            in: {
                              $multiply: [
                                "$$prod.UnitPrice",
```

```
                                    "$$prod.Quantity",
                                    { $subtract: [1, "$$prod.Discount"] }
                                  ]
                                }
                              }
                            }
                          }
                        }
                      ]
                    },
                    // jeśli to inne zamówienie, zostawiamy je w spokoju
                    else: "$$order"
                  }
                }
              }
            }
          }
        }
      ]
    );
```

wyniki

```
// OrdersInfo:
[
  {
    "_id": {"$oid": "6a067a0940cc6835962cdab3"},
    "Customer": {
      "CustomerID": "ALFKI",
      "CompanyName": "Alfreds Futterkiste",
      "City": "Berlin",
      "Country": "Germany"
    },
    "Dates": {
      "OrderDate": {"$date": "2026-05-15T00:00:00.000Z"},
      "RequiredDate": {"$date": "2026-06-15T00:00:00.000Z"}
    },
    "Employee": {
      "EmployeeID": 1,
      "FirstName": "Nancy",
      "LastName": "Davolio",
      "Title": "Sales Representative"
    },
    "Freight": 15.5,
    "OrderID": 99999,
    "OrderTotal": 489.25,
    "Orderdetails": [
      {
        "UnitPrice": 18,
        "Quantity": 20,
        "Discount": 0.05,
```

```
          "Product": {
            "ProductID": 1,
            "ProductName": "Chai",
            "CategoryID": 1,
            "CategoryName": "Beverages"
          }
        },
        {
          "UnitPrice": 31,
          "Quantity": 5,
          "Discount": 0.05,
          "Product": {
            "ProductID": 10,
            "ProductName": "Ikura",
            "CategoryID": 8,
            "CategoryName": "Seafood"
          }
        }
      ],
      "Shipment": {
        "Shipper": {
          "ShipperID": 1,
          "CompanyName": "Speedy Express"
        },
        "ShipName": "Alfreds Futterkiste",
        "ShipAddress": "Obere Str. 57",
        "ShipCity": "Berlin",
        "ShipCountry": "Germany"
      }
    }
  ]

  // CustomerInfo
  [
    {
      "CustomerID": "ALFKI",
      "Orders": [
        {
          "OrderID": 99999,
          "Dates": {
            "OrderDate": {"$date": "2026-05-15T00:00:00.000Z"},
            "RequiredDate": {"$date": "2026-06-15T00:00:00.000Z"}
          },
          "Employee": {
            "EmployeeID": 1,
            "FirstName": "Nancy",
            "LastName": "Davolio",
            "Title": "Sales Representative"
          },
          "Freight": 15.5,
          "OrderTotal": 489.25,
          "Shipment": {
            "Shipper": {
              "ShipperID": 1,
```

```
            "CompanyName": "Speedy Express"
          },
          "ShipName": "Alfreds Futterkiste",
          "ShipAddress": "Obere Str. 57",
          "ShipCity": "Berlin",
          "ShipCountry": "Germany"
        },
        "Orderdetails": [
          {
            "UnitPrice": 18,
            "Quantity": 20,
            "Discount": 0.05,
            "Product": {
              "ProductID": 1,
              "ProductName": "Chai"
            }
          },
          {
            "UnitPrice": 31,
            "Quantity": 5,
            "Discount": 0.05,
            "Product": {
              "ProductID": 10,
              "ProductName": "Ikura"
            }
          }
        ]
      }
    ]
  }
]
```

UWAGA: W raporcie należy zamieścić kod poleceń oraz uzyskany rezultat, np wynik polecenia `db.kolekcka.fimd().limit(2)` lub jego fragment

> [!NOTE] rozwiązania, obok treści zadań.

Zadanie 2 - modelowanie danych

Zaproponuj strukturę bazy danych dla wybranego/przykładowego zagadnienia/problemu

Należy wybrać jedno zagadnienie/problem (A lub B lub C)

Przykład A

- Wykładowcy, przedmioty, studenci, oceny
  - Wykładowcy prowadzą zajęcia z poszczególnych przedmiotów
  - Studenci uczęszczają na zajęcia
  - Wykładowcy wystawiają oceny studentom
  - Studenci oceniają zajęcia

Przykład B

- Firmy, wycieczki, osoby
  - Firmy organizują wycieczki
  - Osoby rezerwują miejsca/wykupują bilety
  - Osoby oceniają wycieczki

Przykład C

- Własny przykład o podobnym stopniu złożoności

a) Zaproponuj różne warianty struktury bazy danych i dokumentów w poszczególnych kolekcjach oraz przeprowadzić dyskusję każdego wariantu (wskazać wady i zalety każdego z wariantów)

- zdefiniuj schemat/reguły walidacji danych
- wykorzystaj referencje
- dokumenty zagnieżdżone
- tablice

b) Kolekcje należy wypełnić przykładowymi danymi

c) W kontekście zaprezentowania wad/zalet należy zaprezentować kilka przykładów/zapytań/operacji oraz dla których dedykowany jest dany wariant

W sprawozdaniu należy zamieścić przykładowe dokumenty w formacie JSON (pkt a) i b)), oraz kod zapytań/operacji (pkt c)), wraz z odpowiednim komentarzem opisującym strukturę dokumentów oraz polecenia ilustrujące wykonanie przykładowych operacji na danych

Do sprawozdania należy dołączyć

- plik z kodem operacji/zapytań w wersji źródłowej (np. plik .js, np. plik .md)
- oraz kompletny zrzut wykonanych/przygotowanych baz danych (taki zrzut można wykonać np. za pomocą poleceń `mongoexport`, `mongdump` …)
  - załącznik ze zrzutem baz powinien mieć format zip

Zadanie 2 - rozwiązanie - Michał Kościanek, code review - Michał Mąka

Wybrane zagadnienie: B - Firmy, wycieczki, osoby

---

a) Warianty struktury bazy danych - analiza i dyskusja

WARIANT 1: Struktura znormalizowana (referencje)

Struktury dokumentów:

```
// Kolekcja: companies
{
  "_id": ObjectId("..."),
```

```
  "companyID": "COMP001",
  "name": "AwesomeTrips",
  "email": "info@awesometrips.com",
  "phone": "+48123456789",
  "address": "ul. Podróżna 10, Warszawa",
  "website": "www.awesometrips.com",
  "foundedYear": 2015,
  "rating": 4.5
}

// Kolekcja: excursions
{
  "_id": ObjectId("..."),
  "excursionID": "EXC001",
  "companyID": "COMP001",
  "title": "Rejs po Dunaju",
  "description": "Wycieczka po Dunaju z wizytą w Wiedniu",
  "destination": "Wiedeń, Austria",
  "startDate": ISODate("2026-06-15"),
  "endDate": ISODate("2026-06-22"),
  "duration": 8,
  "availableSeats": 45,
  "totalSeats": 50,
  "pricePerPerson": 2500,
  "category": "River Cruise",
  "difficulty": "Easy",
  "highlights": ["Dunaj", "Wiedeń", "Bratysława"]
}

// Kolekcja: persons
{
  "_id": ObjectId("..."),
  "personID": "PER001",
  "firstName": "Jan",
  "lastName": "Kowalski",
  "email": "jan.kowalski@email.com",
  "phone": "+48987654321",
  "birthDate": ISODate("1985-03-15"),
  "nationality": "Polish",
  "registrationDate": ISODate("2024-01-10")
}

// Kolekcja: reservations
{
  "_id": ObjectId("..."),
  "reservationID": "RES001",
  "personID": "PER001",
  "excursionID": "EXC001",
  "reservationDate": ISODate("2026-05-01"),
  "numberOfSeats": 2,
  "totalPrice": 5000,
  "status": "Confirmed",
  "paymentStatus": "Paid",
  "paymentDate": ISODate("2026-05-05")
```

```
}

// Kolekcja: reviews
{
  "_id": ObjectId("..."),
  "reviewID": "REV001",
  "personID": "PER001",
  "excursionID": "EXC001",
  "rating": 5,
  "comment": "Wspaniała wycieczka! Polecam wszystkim!",
  "reviewDate": ISODate("2026-06-25"),
  "helpfulCount": 12
}
```

Zalety:

- Brak redundancji danych
- Łatwe aktualizacje (np. zmiana danych firmy w jednym miejscu)
- Zgodne z normalizacją relacyjną
- Oszczędność pamięci

Wady:

- Wiele operacji lookup() wymaganych dla złożonych zapytań
- Gorsza wydajność przy czytaniu
- Konieczność łączenia danych z wielu kolekcji

WARIANT 2: Struktura denormalizowana (dokumenty zagnieżdżone)

Struktury dokumentów:

```
// Kolekcja: excursions_denormalized (Wycieczki z zagnieżdżonymi danymi)
{
  "_id": ObjectId("..."),
  "excursionID": "EXC001",
  "title": "Rejs po Dunaju",
  "description": "Wycieczka po Dunaju z wizytą w Wiedniu",
  "destination": "Wiedeń, Austria",
  "startDate": ISODate("2026-06-15"),
  "endDate": ISODate("2026-06-22"),
  "duration": 8,
  "availableSeats": 45,
  "totalSeats": 50,
  "pricePerPerson": 2500,
  "category": "River Cruise",
  "highlights": ["Dunaj", "Wiedeń", "Bratysława"],
  "company": {
    "companyID": "COMP001",
    "name": "AwesomeTrips",
```

```
      "email": "info@awesometrips.com",
      "phone": "+48123456789",
      "website": "www.awesometrips.com",
      "rating": 4.5
    },
    "reservations": [
      {
        "reservationID": "RES001",
        "person": {
          "personID": "PER001",
          "firstName": "Jan",
          "lastName": "Kowalski",
          "email": "jan.kowalski@email.com",
          "nationality": "Polish"
        },
        "numberOfSeats": 2,
        "totalPrice": 5000,
        "reservationDate": ISODate("2026-05-01"),
        "status": "Confirmed",
        "paymentStatus": "Paid"
      }
    ],
    "reviews": [
      {
        "reviewID": "REV001",
        "personID": "PER001",
        "personName": "Jan Kowalski",
        "rating": 5,
        "comment": "Wspaniała wycieczka!",
        "reviewDate": ISODate("2026-06-25")
      }
    ],
    "averageRating": 4.8,
    "totalReviews": 15
  }
```

Zalety:

- Bardzo szybkie odczyty (jedno zapytanie)
- Kompletne dane w jednym dokumencie
- Łatwiejsze operacje agregacji
- Lepsza wydajność dla scenariuszy read-heavy

Wady:

- Duża redundancja danych
- Trudne aktualizacje (trzeba aktualizować w wielu miejscach)
- Rozmiar dokumentu może się szybko powiększać
- Trudności przy modyfikacji danych osób lub firm

****WARIANT 3: Struktura hybrydowa ****

Struktury dokumentów:

```
// Kolekcja: companies
{
  "_id": ObjectId("..."),
  "companyID": "COMP001",
  "name": "AwesomeTrips",
  "email": "info@awesometrips.com",
  "phone": "+48123456789",
  "address": "ul. Podróżna 10, Warszawa",
  "website": "www.awesometrips.com",
  "rating": 4.5
}

// Kolekcja: persons
{
  "_id": ObjectId("..."),
  "personID": "PER001",
  "firstName": "Jan",
  "lastName": "Kowalski",
  "email": "jan.kowalski@email.com",
  "phone": "+48987654321",
  "nationality": "Polish"
}

// Kolekcja: excursions_hybrid (Wycieczki - z zagnieżdżonymi recenzjami,
ale referencje do osób)
{
  "_id": ObjectId("..."),
  "excursionID": "EXC001",
  "title": "Rejs po Dunaju",
  "description": "Wycieczka po Dunaju z wizytą w Wiedniu",
  "destination": "Wiedeń, Austria",
  "startDate": ISODate("2026-06-15"),
  "endDate": ISODate("2026-06-22"),
  "duration": 8,
  "pricePerPerson": 2500,
  "availableSeats": 45,
  "totalSeats": 50,
  "companyID": "COMP001",
  "companyName": "AwesomeTrips",
  "companyRating": 4.5,
  "category": "River Cruise",
  "highlights": ["Dunaj", "Wiedeń", "Bratysława"],
  "reservations": [
    {
      "reservationID": "RES001",
      "personID": "PER001",
      "personName": "Jan Kowalski",
      "numberOfSeats": 2,
```

```
      "totalPrice": 5000,
      "reservationDate": ISODate("2026-05-01"),
      "status": "Confirmed"
    }
  ],
  "reviews": [
    {
      "reviewID": "REV001",
      "personID": "PER001",
      "personName": "Jan Kowalski",
      "rating": 5,
      "comment": "Wspaniała wycieczka!",
      "reviewDate": ISODate("2026-06-25")
    }
  ],
  "averageRating": 4.8
}
```

Zalety:

- Równowaga między wydajnością a normalizacją
- Szybkie odczyty wycieczek z recenzjami
- Mniej redundancji niż pełna denormalizacja
- Możliwość weryfikacji danych poprzez referencje
- Dobrze skaluje się dla tych konkretnych operacji

Wady:

- Złożoność w implementacji
- Trzeba zarządzać duplikatami danych osób w rezerwacjach
- Wymaga synchronizacji przy zmianach danych

---

b) Populacja kolekcji przykładowymi danymi

```
// 1. Wyczyszczenie baz
db.companies.deleteMany({});
db.persons.deleteMany({});
db.excursions_hybrid.deleteMany({});

// 2. Dodanie firm
db.companies.insertMany([
  {
    "companyID": "COMP001",
    "name": "AwesomeTrips",
    "email": "info@awesometrips.com",
    "phone": "+48123456789",
    "address": "ul. Podróżna 10, Warszawa",
    "website": "www.awesometrips.com",
    "foundedYear": 2015,
```

```
      "rating": 4.5,
      "totalReviews": 150
    },
    {
      "companyID": "COMP002",
      "name": "EuroAdventures",
      "email": "hello@euroadventures.com",
      "phone": "+33123456789",
      "address": "10 Rue de Paris, Lyon",
      "website": "www.euroadventures.com",
      "foundedYear": 2010,
      "rating": 4.3,
      "totalReviews": 200
    },
    {
      "companyID": "COMP003",
      "name": "PolarExpeditions",
      "email": "contact@polarexp.com",
      "phone": "+47987654321",
      "address": "Arctic House, Tromsø",
      "website": "www.polarexp.com",
      "foundedYear": 2018,
      "rating": 4.7,
      "totalReviews": 80
    }
  ]);

  // 3. Dodanie osób
  db.persons.insertMany([
    {
      "personID": "PER001",
      "firstName": "Jan",
      "lastName": "Kowalski",
      "email": "jan.kowalski@email.com",
      "phone": "+48987654321",
      "birthDate": ISODate("1985-03-15"),
      "nationality": "Polish",
      "registrationDate": ISODate("2024-01-10")
    },
    {
      "personID": "PER002",
      "firstName": "Maria",
      "lastName": "Nowak",
      "email": "maria.nowak@email.com",
      "phone": "+48987654322",
      "birthDate": ISODate("1990-07-22"),
      "nationality": "Polish",
      "registrationDate": ISODate("2024-02-15")
    },
    {
      "personID": "PER003",
      "firstName": "Pierre",
      "lastName": "Dubois",
      "email": "pierre.dubois@email.fr",
```

```
      "phone": "+33612345678",
      "birthDate": ISODate("1988-11-05"),
      "nationality": "French",
      "registrationDate": ISODate("2024-03-20")
    },
    {
      "personID": "PER004",
      "firstName": "Anna",
      "lastName": "Schmidt",
      "email": "anna.schmidt@email.de",
      "phone": "+49301234567",
      "birthDate": ISODate("1992-05-30"),
      "nationality": "German",
      "registrationDate": ISODate("2024-04-01")
    }
  ]);

  // 4. Dodanie wycieczek z rezerwacjami i recenzjami
  db.excursions_hybrid.insertMany([
    {
      "excursionID": "EXC001",
      "title": "Rejs po Dunaju",
      "description": "8-dniowa wycieczka po Dunaju z wizytą w Wiedniu,
  Bratysławie i Budapeszcie",
      "destination": "Austria, Słowacja, Węgry",
      "startDate": ISODate("2026-06-15"),
      "endDate": ISODate("2026-06-22"),
      "duration": 8,
      "pricePerPerson": 2500,
      "availableSeats": 42,
      "totalSeats": 50,
      "companyID": "COMP001",
      "companyName": "AwesomeTrips",
      "companyRating": 4.5,
      "category": "River Cruise",
      "difficulty": "Easy",
      "highlights": ["Dunaj", "Wiedeń", "Bratysława", "Budapeszt"],
      "accommodationType": "4-star hotel",
      "mealsPlan": "Half Board",
      "reservations": [
        {
          "reservationID": "RES001",
          "personID": "PER001",
          "personName": "Jan Kowalski",
          "numberOfSeats": 2,
          "totalPrice": 5000,
          "reservationDate": ISODate("2026-05-01"),
          "status": "Confirmed",
          "paymentStatus": "Paid",
          "paymentDate": ISODate("2026-05-05")
        },
        {
          "reservationID": "RES002",
          "personID": "PER002",
```

```
          "personName": "Maria Nowak",
          "numberOfSeats": 1,
          "totalPrice": 2500,
          "reservationDate": ISODate("2026-05-03"),
          "status": "Confirmed",
          "paymentStatus": "Paid",
          "paymentDate": ISODate("2026-05-08")
        }
      ],
      "reviews": [
        {
          "reviewID": "REV001",
          "personID": "PER001",
          "personName": "Jan Kowalski",
          "rating": 5,
          "comment": "Wspaniała wycieczka! Rewelacyjna organizacja i piękne
widoki. Gorąco polecam!",
          "reviewDate": ISODate("2026-06-25")
        },
        {
          "reviewID": "REV002",
          "personID": "PER002",
          "personName": "Maria Nowak",
          "rating": 4,
          "comment": "Bardzo dobra wycieczka, jedynym minusem było idójó
porankami.",
          "reviewDate": ISODate("2026-06-26")
        }
      ],
      "averageRating": 4.5,
      "totalReviews": 2,
      "createdDate": ISODate("2026-01-10")
    },
    {
      "excursionID": "EXC002",
      "title": "Eksploracja Grónjlandii",
      "description": "10-dniowa arktyczna eksploracja Grenlandii z obserwacją
lodowców i aurory",
      "destination": "Grenlandia",
      "startDate": ISODate("2026-08-20"),
      "endDate": ISODate("2026-08-30"),
      "duration": 10,
      "pricePerPerson": 5500,
      "availableSeats": 18,
      "totalSeats": 20,
      "companyID": "COMP003",
      "companyName": "PolarExpeditions",
      "companyRating": 4.7,
      "category": "Adventure",
      "difficulty": "Hard",
      "highlights": ["Lodowce", "Aurora Borealis", "Inuici", "Nuuk"],
      "accommodationType": "Luxury lodge",
      "mealsPlan": "Full Board",
      "reservations": [
```

```
      {
        "reservationID": "RES003",
        "personID": "PER003",
        "personName": "Pierre Dubois",
        "numberOfSeats": 1,
        "totalPrice": 5500,
        "reservationDate": ISODate("2026-05-10"),
        "status": "Confirmed",
        "paymentStatus": "Paid",
        "paymentDate": ISODate("2026-05-15")
      }
    ],
    "reviews": [
      {
        "reviewID": "REV003",
        "personID": "PER003",
        "personName": "Pierre Dubois",
        "rating": 5,
        "comment": "Niesamowite doświadczenie! Polecam każdemu miłośnikowi
przyrody!",
        "reviewDate": ISODate("2026-08-31")
      }
    ],
    "averageRating": 5.0,
    "totalReviews": 1,
    "createdDate": ISODate("2026-02-01")
  },
  {
    "excursionID": "EXC003",
    "title": "Paryż - Miasto Światła",
    "description": "5-dniowa wycieczka po Paryżu z wizytą w Luwrze, Notre-
Dame i Wieży Eiffla",
    "destination": "Francja",
    "startDate": ISODate("2026-07-10"),
    "endDate": ISODate("2026-07-14"),
    "duration": 5,
    "pricePerPerson": 1800,
    "availableSeats": 35,
    "totalSeats": 40,
    "companyID": "COMP002",
    "companyName": "EuroAdventures",
    "companyRating": 4.3,
    "category": "Cultural",
    "difficulty": "Easy",
    "highlights": ["Wieża Eiffla", "Luwr", "Notre-Dame", "Pałac
Wersalski"],
    "accommodationType": "3-star hotel",
    "mealsPlan": "Breakfast",
    "reservations": [
      {
        "reservationID": "RES004",
        "personID": "PER004",
        "personName": "Anna Schmidt",
        "numberOfSeats": 2,
```

```
        "totalPrice": 3600,
        "reservationDate": ISODate("2026-05-20"),
        "status": "Confirmed",
        "paymentStatus": "Pending"
      }
    ],
    "reviews": [
      {
        "reviewID": "REV004",
        "personID": "PER001",
        "personName": "Jan Kowalski",
        "rating": 4,
        "comment": "Bardzo ładna wycieczka, choć trochę zbyt pospieszona.",
        "reviewDate": ISODate("2026-07-15")
      }
    ],
    "averageRating": 4.0,
    "totalReviews": 1,
    "createdDate": ISODate("2026-01-20")
  }
]);
```

---

c) Przykładowe operacje i zapytania

Zapytanie 1: Wycieczki z liczbą dostępnych miejsc > 30

```
db.excursions_hybrid.find(
  { availableSeats: { $gt: 30 } },
  { title: 1, destination: 1, availableSeats: 1, pricePerPerson: 1 }
);

// WYNIK:
[
  {
    "_id": ObjectId("..."),
    "title": "Rejs po Dunaju",
    "destination": "Austria, Słowacja, Węgry",
    "availableSeats": 42,
    "pricePerPerson": 2500
  },
  {
    "_id": ObjectId("..."),
    "title": "Paryż - Miasto Światła",
    "destination": "Francja",
    "availableSeats": 35,
    "pricePerPerson": 1800
  }
]
```

Analiza: W wariancie hybrydowym takie zapytanie jest szybkie, bo dane znajdują się w jednym dokumencie.

Zapytanie 2: Przychód z każdej wycieczki (suma rezerwacji)

```
db.excursions_hybrid.aggregate([
  {
    $project: {
      title: 1,
      companyName: 1,
      totalRevenue: {
        $sum: "$reservations.totalPrice"
      },
      numberOfReservations: {
        $size: "$reservations"
      },
      occupancyRate: {
        $round: [
          {
            $multiply: [
              {
                $divide: [
                  { $subtract: ["$totalSeats", "$availableSeats"] },
                  "$totalSeats"
                ]
              },
              100
            ]
          },
          2
        ]
      }
    }
  },
  {
    $sort: { totalRevenue: -1 }
  }
]);

// WYNIK:
[
  {
    "_id": ObjectId("..."),
    "title": "Eksploracja Grenlandii",
    "companyName": "PolarExpeditions",
    "totalRevenue": 5500,
    "numberOfReservations": 1,
    "occupancyRate": 5
  },
  {
    "_id": ObjectId("..."),
```

```
    "title": "Rejs po Dunaju",
    "companyName": "AwesomeTrips",
    "totalRevenue": 7500,
    "numberOfReservations": 2,
    "occupancyRate": 4
  },
  {
    "_id": ObjectId("..."),
    "title": "Paryż - Miasto Światła",
    "companyName": "EuroAdventures",
    "totalRevenue": 3600,
    "numberOfReservations": 1,
    "occupancyRate": 2.5
  }
]
```

Analiza: Zagnieżdżone rezerwacje umożliwiają łatwe obliczenie przychodów bez dodatkowych lookup().

Zapytanie 3: Wycieczki oceniane na 4+ gwiazdki z recenzjami

```
db.excursions_hybrid.find(
  { averageRating: { $gte: 4.0 } },
  {
    title: 1,
    averageRating: 1,
    totalReviews: 1,
    "reviews": {
      $filter: {
        input: "$reviews",
        as: "review",
        cond: { $gte: ["$$review.rating", 4] }
      }
    }
  }
);

// WYNIK:
[
  {
    "_id": ObjectId("..."),
    "title": "Rejs po Dunaju",
    "averageRating": 4.5,
    "totalReviews": 2,
    "reviews": [
      {
        "reviewID": "REV001",
        "personID": "PER001",
        "personName": "Jan Kowalski",
        "rating": 5,
```

```
        "comment": "Wspaniała wycieczka!"
      },
      {
        "reviewID": "REV002",
        "personID": "PER002",
        "personName": "Maria Nowak",
        "rating": 4,
        "comment": "Bardzo dobra wycieczka..."
      }
    ]
  },
  {
    "_id": ObjectId("..."),
    "title": "Eksploracja Grenlandii",
    "averageRating": 5.0,
    "totalReviews": 1,
    "reviews": [...]
  }
]
```

Zapytanie 4: Wycieczki danej firmy z ponad 30% okupancją

```
db.excursions_hybrid.aggregate([
  {
    $match: { companyID: "COMP001" }
  },
  {
    $addFields: {
      occupancyRate: {
        $multiply: [
          {
            $divide: [
              { $subtract: ["$totalSeats", "$availableSeats"] },
              "$totalSeats"
            ]
          },
          100
        ]
      }
    }
  },
  {
    $match: { occupancyRate: { $gte: 30 } }
  },
  {
    $project: {
      title: 1,
      destination: 1,
      occupancyRate: { $round: ["$occupancyRate", 2] },
      averageRating: 1
```

```
    }
  }
]);

// WYNIK:
[
  {
    "_id": ObjectId("..."),
    "title": "Rejs po Dunaju",
    "destination": "Austria, Słowacja, Węgry",
    "occupancyRate": 16.0,
    "averageRating": 4.5
  }
]
```

Zapytanie 5: Osoby, które rezerwowały wiele wycieczek (z lookup)

```
db.excursions_hybrid.aggregate([
  {
    $unwind: "$reservations"
  },
  {
    $group: {
      _id: "$reservations.personID",
      personName: { $first: "$reservations.personName" },
      numberOfReservations: { $sum: 1 },
      totalSpent: { $sum: "$reservations.totalPrice" },
      excursionsTitles: { $push: "$title" }
    }
  },
  {
    $match: { numberOfReservations: { $gt: 1 } }
  },
  {
    $sort: { totalSpent: -1 }
  }
]);

// WYNIK:
[
  {
    "_id": "PER001",
    "personName": "Jan Kowalski",
    "numberOfReservations": 2,
    "totalSpent": 7500,
    "excursionsTitles": ["Rejs po Dunaju", "Paryż - Miasto Światła"]
  }
]
```

Zapytanie 6: Dodanie nowej recenzji do wycieczki

```
db.excursions_hybrid.updateOne(
  { excursionID: "EXC001" },
  {
    $push: {
      reviews: {
        reviewID: "REV005",
        personID: "PER004",
        personName: "Anna Schmidt",
        rating: 5,
        comment: "Cudowna wycieczka! Nie mogę się doczekać następnej!",
        reviewDate: ISODate("2026-06-27")
      }
    },
    $set: {
      averageRating: 4.67,
      totalReviews: 3
    }
  }
);
```

Zapytanie 7: Aktualizacja liczby dostępnych miejsc po rezerwacji

```
db.excursions_hybrid.updateOne(
  { excursionID: "EXC002" },
  {
    $push: {
      reservations: {
        reservationID: "RES005",
        personID: "PER002",
        personName: "Maria Nowak",
        numberOfSeats: 1,
        totalPrice: 5500,
        reservationDate: ISODate("2026-05-25"),
        status: "Confirmed",
        paymentStatus: "Paid",
        paymentDate: ISODate("2026-05-26")
      }
    },
    $inc: { availableSeats: -1 }
  }
);
```

Porównanie wariantów - podsumowanie:

Operacja	Wariant 1 (Normalizowany)	Wariant 2 (Denormalizowany)	Wariant 3 (Hybrydowy)
Pobierz wycieczki z recenzjami	3-5 lookup()	1 zapytanie	1 zapytanie
Oblicz przychód	Złożona agregacja	Proste	Proste
Zmiana danych osoby	1 update	Wiele updates	Wymaga synchronizacji
Rozmiar dokumentu	Mały	Ogromny	Średni
Szybkość czytania	Wolna	Bardzo szybka	Szybka
Spójność danych	Wysoka	Niska	Średnia

Wniosek: Wariant 3 oferuje najlepszy balans między wydajnością a strukturą danych dla tego scenariusza.